Possible Code Optimizations for my-litrpg-gameztrui

Balanced runtime, maintainability, and delivery audit

Prepared: March 10, 2026
Scope: React/Vite frontend, game-core systems, Firebase Functions runtime, content registries, and tooling pipeline.
Output: A 20-page repo-specific optimization report with prioritized recommendations.

This report reviews the current codebase through a balanced lens. It does not treat optimization as a synonym for micro-performance tuning. Instead, it looks at the places where runtime cost, cognitive load, bundle weight, and engineering friction reinforce each other and create compounding drag.

The strongest theme in this repository is concentration. A relatively small set of orchestration files carries a large share of business logic, data loading, and state transition complexity. That means the highest-value improvements are likely to come from architectural decomposition, selective loading, and clearer ownership boundaries rather than from isolated low-level rewrites.

Metric	Observed value
Audited code files	310
Approximate code lines	75,847
Top 10 hotspot share	26.7%
Top 25 hotspot share	47.2%
Root npm scripts	68
Test files	68
Dynamic imports in src	4

Recommendation in one sentence: optimize the repo by shrinking orchestration surfaces, compiling or lazily loading static content, and turning high-branching systems into smaller deterministic modules with measurable contracts.

Table of Contents and Repo Snapshot

Page map and the most important structural facts

Contents

- Pages 1-4: Summary, structure, and audit method.
- Pages 5-8: Frontend orchestration, narrative flow, and bundle strategy.
- Pages 9-12: Game-core hotspots, combat decomposition, and content data.
- Pages 13-17: Functions, tooling, test strategy, and roadmap.
- Pages 18-20: Appendices and closing notes.

The repository is not a narrow single-surface application. It combines a React frontend under **src/**, a large shared simulation and content layer under **packages/game-core/src/**, and operational server logic under **functions/lib/**. That structure is healthy in intent, but several runtime paths still cut across these boundaries in a way that causes large files and large bundles.

The code scan covered **310** source files and about **75,847** lines of executable code. The top ten files carry **26.7%** of that footprint, which is high enough to treat hotspot concentration as a primary design risk.

- Largest hotspot: CombatSystem.js at 3,594 lines, holding major combat orchestration.
- Largest frontend hotspot: GameUI.jsx at 1,295 lines, coordinating multiple gameplay surfaces.
- Largest narrative hotspot: useNarrative.js at 1,844 lines, mixing prompt, cache, and quest concerns.

Executive Summary

The highest-value changes to pursue first

The codebase already demonstrates useful discipline in a few areas: the app and functions projects both typecheck cleanly, Vite manual chunking is in place, and core gameplay logic has been centralized rather than scattered across many thin files. Those are solid foundations. The next stage of optimization should focus on turning those centralized systems into smaller, cheaper-to-load, and easier-to-test modules.

- Highest ROI: split orchestration-heavy files such as GameUI, useNarrative, gameReducer, and CombatSystem along domain seams instead of function-by-function.
- Highest runtime leverage: reduce eager shipping of large game-core skill and archetype payloads that currently contribute to the biggest browser chunks.
- Highest workflow leverage: narrow lint/build/test scope so generated artifacts do not inflate feedback cycles or create noisy false work.
- Most important safeguard: add measurable baselines for chunk size, combat-step time, and reducer/action latency before deep refactors begin.

If the team can only fund a small optimization wave, the recommendation is to spend it on three moves: carve GameUI into feature controllers and presentational layers, compile static registries into demand-loaded slices, and extract combat rule families out of CombatSystem behind explicit interfaces. Those changes will improve both performance and maintainability without forcing a rewrite.

Priority	Finding	Why it is urgent
P1	Monolithic orchestration files	They slow every future change and hide localized wins.
P1	Large gameplay bundles	They increase startup cost and keep too much content on the critical path.
P2	Registry-heavy static data	Large JSON payloads reduce flexibility and waste memory.
P2	Tooling scope creep	Generated folders are being linted, which erodes developer throughput.
P3	Testing is broad but not enough baseline driven	Optimization work needs stronger before/after signals.

Audit Method and System Map

How the findings were derived

This audit combined direct file inspection, simple structural metrics, and light tooling validation. It is intentionally practical: the goal is to highlight optimizations that matter in this repository as it exists today, not to produce an academic complexity scorecard.

- Scanned source-owned JS/JSX/TS/TSX files in src, packages/game-core/src, and functions/lib.
- Measured hotspot concentration by file line counts and local structural markers.
- Built the web app to inspect current chunk composition under dist/assets.
- Ran root and functions typecheck to separate optimization work from correctness debt.
- Ran lint and observed generated-artifact scope leakage in the root workspace.

The system map that emerges is straightforward. The frontend shell coordinates auth, save flows, character setup, and gameplay presentation. Game-core supplies the bulk of deterministic logic, content registries, simulation helpers, and combat rules. Functions add sync and content-generation responsibilities. Optimization opportunity appears where these layers remain coupled through large orchestration surfaces.

Layer	Current role	Optimization lens
src/	UI shell, hooks, feature modes, and client services	Reduce rerenders, shrink lazy chunks, simplify orchestration.
packages/game-core/src/	Rules, registries, combat, simulation, and content	Split large engines, compile static data, localize ownership.
functions/lib/	Sync, API routing, generation, and persistence	Separate hot path logic from adapters and storage concerns.

Frontend Shell and UI Orchestration

Why GameUI is a prime maintainability and runtime target

src/components/GameUI.jsx is roughly **1295** lines long, uses **15** local state hooks, **5** effects, **13** callbacks, and **18** inline style blocks. That combination usually signals an orchestration component that has taken on multiple roles: session control, save management, transition control, and gameplay presentation.

Large UI shells create two kinds of drag. First, they increase rerender probability because many state changes terminate in the same component tree. Second, they make safe extraction harder because domain logic and display concerns are woven together.

- Split GameUI into feature controllers: save/resume, transition shell, gameplay surface, and utility overlays.
- Move inline style blocks into CSS modules or themed utility classes for smaller JSX payloads and easier memoization.
- Introduce selector-style derivation functions so child components receive stable, pre-shaped props.
- Consider mode-based subtrees so inactive gameplay features do not stay mounted or imported together.

A good target state is a thin top-level gameplay shell whose responsibilities are coordination and suspense boundaries, not business logic ownership. That change alone would make later performance work far cheaper.

Narrative Pipeline

The useNarrative hook is doing too many jobs at once

src/hook/useNarrative.js is about **1844** lines and combines prompt assembly, AI calls, scene seeding, narrative cache behavior, opening-scene templates, quest side effects, and user-facing suggestion logic. That makes it both a latency-sensitive runtime path and a high-maintenance code surface.

- Extract prompt construction into pure deterministic builders with explicit input contracts.
- Separate narrative transport from narrative state; AI call orchestration should not live inside content templates.
- Move opening-scene templates and derived quest data into generated content packs or dedicated registries.
- Keep cache policy in a focused adapter so invalidation rules can be tested independently.

This matters for both speed and correctness. Narrative systems often become optimization blind spots because they look asynchronous by nature. In reality, a large share of their cost is deterministic data shaping done before the network call ever starts.

Concern	Current shape	Better shape
Prompt building	Interleaved with hook logic	Pure builder module with typed inputs
Template ownership	Large inline maps	Generated or imported content packs
Cache policy	Implicit in hook	Dedicated cache adapter with tests
Side effects	Mixed into narrative flow	Explicit outcome pipeline

Rendering and State Efficiency

Reduce needless work before chasing micro-optimizations

React performance work is most effective when the component graph is shaped well. In the current repo, the biggest likely win is not sprinkling memoization everywhere. It is reducing the amount of cross-cutting state that reaches large subtrees.

- Define explicit view models for gameplay panels so heavy formatting and derivation happen once per action, not per child render.
- Prefer feature-local state machines for creation, save prompts, and overlays instead of many sibling booleans in one shell.
- Use suspense and lazy boundaries at mode or feature level, not only at App.jsx entry points.
- Reserve memoization for expensive derived data after prop-shape cleanup is in place.

The current source tree only exposes **2** lazy boundaries in frontend source and **4** dynamic imports overall. That is a strong sign that the app still loads most capability behind a small number of gates. For a feature-rich LitRPG surface, there is room to slice more intentionally.

A practical next step is to choose one subdomain, such as save/load or character creation, and prove that a narrower state boundary lowers rerender pressure without hurting usability. That gives the team a reusable extraction pattern.

Bundle and Loading Strategy

Manual chunking helps, but content is still arriving too eagerly

The build output shows clear progress: vendor code is split, React is isolated, and game-core skill and archetype chunks are already separated. That is good evidence that bundle size is being managed intentionally. Even so, the largest domain chunks remain large enough to justify deeper changes in how content is loaded.

Built asset	Size	Interpretation
game-core-skills-CRJo2hre.js	592.8 KB	Heavy skill payload suggests eager loading of large JSON-backed registries.
GameUI-Bzx1EKGm.js	529.0 KB	UI orchestration remains a large lazy chunk despite manual chunking.
game-core-archetypes-DH4kyIUN.js	309.2 KB	Archetype data is large enough to justify demand-based loading or precomputed i
vendor-D6qKj9ID.js	223.8 KB	Keep vendor and base app bundles stable while carving domain chunks smaller.
vendor-react-DiLERqYk.js	184.3 KB	Keep vendor and base app bundles stable while carving domain chunks smaller.
index-azGVyXfN.js	100.4 KB	Keep vendor and base app bundles stable while carving domain chunks smaller.

- Move large skill packs behind feature-aware imports or compile them into smaller ownership-based bundles.
- Precompute lightweight indexes for UI selection and search screens rather than importing full raw payloads.
- Track raw and gzip budgets in CI so chunk regressions are visible before they ship.
- Review whether archetype and world-data consumers need full fidelity on initial gameplay load.

A common failure mode in content-heavy games is to keep optimizing Rollup while leaving the content graph unchanged. Past a certain point, the better move is to reduce how much domain data any one route or mode needs to know about up front.

Game-Core Hotspot Map

The shared simulation layer is where complexity concentrates most heavily

The shared game-core package contains about 46,613 lines across 114 source files, which means it holds the majority of executable logic in the repo. That is expected for a systems-heavy game, but the concentration profile deserves attention: large portions of combat, skills, registries, and normalization are each dominated by single large files.

Hotspot file	Lines	Why it matters
packages/game-core/src/core/gameReducer.js	1378	Reducer owns many state transitions and quest/combat coordination point
packages/game-core/src/simulation/combat/ActionResolver.js	1335	Large file size increases cognitive load and slows isolated optimization wo
functions/lib/sync-core.js	1335	Server sync logic carries routing, persistence, and deterministic replay con
src/components/GameUI.jsx	1295	UI shell coordinates auth, saves, overlays, and gameplay presentation.
packages/game-core/src/core/StateNormalizer.js	1147	Large file size increases cognitive load and slows isolated optimization wo
packages/game-core/src/systems/skills/SkillRegistry.js	1130	Large file size increases cognitive load and slows isolated optimization wo
functions/lib/bulk-generation-service.js	1125	Large file size increases cognitive load and slows isolated optimization wo
functions/lib/sync.js	1031	Server sync logic carries routing, persistence, and deterministic replay con

gameReducer.js alone carries **31** switch cases and **17** imports. That is a classic sign that command handling, domain policy, and side-effect orchestration are sharing a reducer surface that will keep growing as new systems land.

- Use reducer slices or command-handler registries so systems can evolve with narrower ownership.
- Keep pure state transitions separate from post-action enrichment such as narrative and rewards.
- Make data-normalization passes explicit and measurable so they do not become hidden tax on every action.

Combat Engine Optimization Plan

CombatSystem is the strongest candidate for structural decomposition

CombatSystem.js is about **3594** lines with **27** imports, **14** exported functions, and roughly **295** conditional branches. That makes it both a runtime hotspot and a design hotspot. When a file does too much combat orchestration, every rule change becomes a regression risk.

- Split combat into phases such as targeting, cost resolution, damage resolution, effect application, and end-of-turn reconciliation.
- Replace broad shared state mutation patterns with typed combat context objects passed between phase modules.
- Pre-resolve race, title, and equipment modifiers into a compact combat snapshot before action resolution starts.
- Move combat log string construction out of the hot path or into append-only event formatters.

Module family	What moves out of CombatSystem
Preparation	Target validation, lane rules, action legality, and snapshot shaping
Resolution	Damage math, crit rules, bypasses, cooldown and cost changes
Effects	Status application, charm, stealth, item procs, and follow-up triggers
Aftermath	XP, drops, progression updates, and combat log formatting

This is not a call for over-abstraction. The goal is to group rules by lifecycle stage so the team can optimize and test a family of mechanics without scanning thousands of lines.

Data and Content Pipeline

Static payloads should be easier to compile, index, and ship selectively

The repository ships large JSON-backed content sets for skills, archetypes, authorities, and other gameplay metadata. Static content is a strength, but raw payload size and import shape matter. When registries import everything eagerly, they increase both bundle weight and in-memory working set.

Data asset	Size	Optimization angle
packages/game-core/src/data/archetypes/systems.json	229.9 KB	Split into thematic shards or compile to indexed lookup maps.
packages/game-core/src/data/skills/non_combat.json	199.4 KB	Load on demand from tooling screens instead of startup-critical paths.
packages/game-core/src/data/archetypes/authorities.json	69.6 KB	Precompute summaries used by the UI instead of shipping full raw records
packages/game-core/src/data/skills/universal.json	51.1 KB	Consider generated indexes or server-side packaging by feature set.
packages/game-core/src/data/skills/class_abilities.json	37.4 KB	Consider generated indexes or server-side packaging by feature set.
packages/game-core/src/data/skills/race_abilities.json	32.8 KB	Consider generated indexes or server-side packaging by feature set.
packages/game-core/src/data/archetypes/frameworks.json	30.0 KB	Consider generated indexes or server-side packaging by feature set.
packages/game-core/src/data/talent.registry.json	27.2 KB	Consider generated indexes or server-side packaging by feature set.

- Add a build-time content compiler that emits lightweight indexes plus feature-specific shards.
- Separate authoring format from runtime format; rich JSON can stay for editing while compiled artifacts drive the app.
- Cache normalized registry lookups rather than repeating linear scans through large arrays.
- Ship only the content families a mode needs, especially for creation and tooling screens.

A content compiler is especially attractive here because it improves more than performance. It also creates a single place to enforce schema quality, derived indexes, and compatibility checks for evolving class or race systems.

Determinism, Simulation, and State Normalization

Keep deterministic behavior explicit and cheap

The project leans on deterministic gameplay concepts: seeded generation, replay behavior, state normalization, and sync validation. Those are valuable capabilities, but they can become expensive if normalization and simulation preparation happen too frequently or too late in the request path.

- Measure normalization cost separately from action resolution cost so reducers do not hide structural tax.
- Create immutable, pre-shaped snapshots for simulation bridges instead of re-deriving broad state repeatedly.
- Treat replay and sync hash generation as first-class observability signals with baseline budgets.
- Where possible, move deterministic derived data to write-time instead of recomputing it on each read.

A good heuristic is to ask whether each normalization or replay-related step is truly required for the hot path it sits on. Deterministic correctness is essential, but it should not force every action to pay for work that only specific modes or diagnostics need.

Concern	Optimization move
State normalization	Stage it by action type or normalize only changed slices.
Simulation bridge setup	Cache read-only prepared views for repeated combat loops.
Replay hashing	Generate checkpoints at stable boundaries instead of after every micro-step.
Derived quest checks	Use event-specific predicates rather than broad post-action scans.

Functions and Sync Architecture

Server-side logic is smaller, but still worth simplifying

The Firebase Functions surface is smaller than the frontend and game-core layers, but it still contains meaningful hotspots. **functions/lib/sync-core.js** is about **1335** lines and mixes canonicalization, hashing, manifest handling, event normalization, and reward policy concerns. That is enough responsibility density to justify decomposition.

- Split sync-core into event normalization, deterministic hashing, manifest access, and reward policy modules.
- Keep file system and environment access in thin adapters so pure sync logic remains test-friendly.
- Promote canonical serialization and RNG helpers into a focused deterministic utilities layer.
- Review sync.js and bulk-generation paths for repeated parsing or storage work that can be cached per request.

The goal on the server side is not only throughput. It is also confidence. When sync logic is cleanly separated from adapters, deterministic behavior is easier to replay locally and safer to optimize.

Hotspot file	Lines	Why it matters
packages/game-core/src/world/WorldDataRegistry.js	1022	Large file size increases cognitive load and slows isolated optimization wo
src/ui/modes/CharacterCreationMode.jsx	1002	Large file size increases cognitive load and slows isolated optimization wo
src/hook/useCharacterCreationReworked.js	978	Large file size increases cognitive load and slows isolated optimization wo
packages/game-core/src/data/registries/ItemDataRegistry.js	930	Large file size increases cognitive load and slows isolated optimization wo
functions/lib/bulk-gen-store.js	960	Large file size increases cognitive load and slows isolated optimization wo
packages/game-core/src/systems/archetype/ArchetypeSystemRegistry.js	914	Large file size increases cognitive load and slows isolated optimization wo

Backend Runtime Improvements

Small server changes can improve resilience and observability

Functions optimization does not need to start with scaling assumptions. In a repo like this, the first wins usually come from cleaner hot paths, reduced incidental I/O, and better visibility into deterministic runtime behavior.

- Cache manifest and reference data once per cold start with explicit invalidation hooks for local development.
- Move crypto-heavy or serialization-heavy helpers behind measured utilities so their cost is visible in logs.
- Use per-route latency logging around sync and generation handlers to identify whether CPU or storage dominates.
- Standardize payload shaping between router and handler layers to reduce repeated validation work.

If traffic grows, these improvements also create a better foundation for concurrency decisions. Without good decomposition, scaling tends to amplify noisy bottlenecks instead of clarifying them.

Path	Likely cost center	Recommended probe
Sync replay and hashing	Serialization plus canonical hashing	Measure payload size against latency
Bulk generation	I/O and content shaping	Log item counts and generation stages
API router	Request normalization and dispatch	Track route-level latency percentiles

Tooling and CI Efficiency

Developer feedback loops can be tightened with low risk

The root workspace currently defines **68** npm scripts. That reflects healthy operational maturity, but it also signals script sprawl and a need for clear guardrails around what belongs in the hot developer loop versus release-only workflows.

One immediately actionable issue surfaced during the audit: root lint scanned generated artifacts under out/production because the global ignore list covers dist, build, and coverage but does not currently ignore out. That is a small config detail with outsized impact on feedback noise.

- Add out and other generated product directories to global ESLint ignores, or lint only source-owned roots.
- Separate fast local verification from release and rollout scripts so contributors know the cheapest safe loop.
- Publish chunk-size and line-count hotspot reports as generated artifacts instead of manual detective work.
- Consider script namespaces or task runners for related release tracks to reduce command discovery friction.

Loop type	Recommended contents
Inner loop	lint scoped to source roots, typecheck, smoke tests, fast build budget check
Pre-merge	full check, bundle budget enforcement, targeted replay and rules tests
Release	rollout reports, endpoint probes, soak and rollback scripts

Testing Optimization Strategy

Use the test suite to protect optimization work, not just regressions

The repository already has about **68** test files, which is a good base. The next opportunity is to turn that coverage into a better optimization safety net. Performance-sensitive refactors need before-and-after baselines, not only correctness assertions.

- Add micro-bench or timing probes around combat resolution, reducer dispatch, and registry normalization.
- Snapshot bundle sizes and fail builds only on meaningful regression thresholds, not raw churn.
- Create contract tests for extracted sub-engines before large file decomposition begins.
- Use deterministic replay fixtures to prove that combat and sync refactors preserve canonical outcomes.

Optimization efforts often stall because teams cannot prove they are still correct. In this codebase, replay-oriented infrastructure is already present, which means the project is unusually well positioned to make structural changes safely if those fixtures are formalized.

Focus area	Most useful test form
GameUI decomposition	Interaction tests around save, auth, and mode transitions
Narrative pipeline	Pure builder tests plus cache invalidation tests
Combat engine	Deterministic scenario fixtures and timing probes
Registry compiler	Schema validation and output-compatibility snapshots

Phased Roadmap

Sequence the work so every wave unlocks the next

The safest optimization roadmap is not the one that attacks the biggest file first. It is the one that creates observability and extraction patterns before deep surgery begins. The proposed sequence below is designed to generate confidence early while still targeting the highest-value hotspots.

Wave	Scope	Expected effect
0-2 weeks	Lint scope cleanup, chunk telemetry, hotspot baselines	Faster feedback loops and safer refactors
2-6 weeks	Decompose GameUI and narrative pipeline, add selectors	Lower render cost and easier testability
6-10 weeks	Extract combat sub-engines and data compilers	Lower branch density and improved iteration speed
10+ weeks	Formalize domain boundaries and generated content packs	Sustainable scaling for new systems and content

- Begin by making problems visible: tighten lint scope, record chunk sizes, and baseline combat plus reducer timing.
- Use GameUI and useNarrative as the first extraction exemplars because they improve team velocity quickly.
- Only then move into CombatSystem and registry compilation, which are higher leverage but also higher risk.
- Keep every wave independently shippable so optimization does not become a long-lived branch of hidden work.

Appendix A: Hotspot Reference

Largest code files and their likely optimization role

This appendix groups the largest source files because they represent the highest-value review surface for future optimization work. Large files are not automatically bad, but in this repository they align closely with cross-cutting ownership.

Hotspot file	Lines	Why it matters
packages/game-core/src/systems/combat/CombatSystem.js	3594	Combat orchestration, branching, and deterministic state writes sit togethe
packages/game-core/src/systems/skills/TalentDefinitions.js	2973	Large file size increases cognitive load and slows isolated optimization wo
packages/game-core/src/systems/magic/EssenceWeavingSystem.js	2393	Large file size increases cognitive load and slows isolated optimization wo
packages/game-core/src/data/registries/RaceDataRegistry.js	2408	Large file size increases cognitive load and slows isolated optimization wo
src/hook/useNarrative.js	1844	Narrative state, AI orchestration, scene seeding, and caching are coupled.
packages/game-core/src/simulation/validation/schema.ts	1507	Large file size increases cognitive load and slows isolated optimization wo
packages/game-core/src/systems/profession/ProfessionRegistry.js	1428	Large file size increases cognitive load and slows isolated optimization wo
src/components/StatusOverlay.jsx	1382	Large file size increases cognitive load and slows isolated optimization wo
packages/game-core/src/core/gameReducer.js	1378	Reducer owns many state transitions and quest/combat coordination point
packages/game-core/src/simulation/combat/ActionResolver.js	1385	Large file size increases cognitive load and slows isolated optimization wo
functions/lib/sync-core.js	1335	Server sync logic carries routing, persistence, and deterministic replay con
src/components/GameUI.jsx	1295	UI shell coordinates auth, saves, overlays, and gameplay presentation.

The concentration pattern is strong enough that formal file ownership and review checklists would help. When multiple systems converge in one file, reviews should verify both domain correctness and cost growth.

Appendix B: Build and Content Evidence

Concrete payload indicators behind the recommendations

This appendix captures the two evidence sets most relevant to runtime and startup costs: built asset sizes and the heaviest raw content payloads under packages/game-core/src/data.

Built asset	Size	Interpretation
game-core-skills-CRJo2hre.js	592.8 KB	Heavy skill payload suggests eager loading of large JSON-backed registries.
GameUI-Bzx1EKGm.js	529.0 KB	UI orchestration remains a large lazy chunk despite manual chunking.
game-core-archetypes-DH4kyIUN.js	309.2 KB	Archetype data is large enough to justify demand-based loading or precomputed i
vendor-D6qKj9ID.js	223.8 KB	Keep vendor and base app bundles stable while carving domain chunks smaller.
vendor-react-DiLERqYk.js	184.3 KB	Keep vendor and base app bundles stable while carving domain chunks smaller.
index-azGVyXfN.js	100.4 KB	Keep vendor and base app bundles stable while carving domain chunks smaller.
index-DnVTWMPe.css	32.3 KB	Keep vendor and base app bundles stable while carving domain chunks smaller.
game-core-world-data-C37b4GfR.js	17.2 KB	Keep vendor and base app bundles stable while carving domain chunks smaller.

Data asset	Size	Optimization angle
packages/game-core/src/data/archetypes/systems.json	229.9 KB	Split into thematic shards or compile to indexed lookup maps.
packages/game-core/src/data/skills/non_combat.json	199.4 KB	Load on demand from tooling screens instead of startup-critical paths.
packages/game-core/src/data/archetypes/authorities.json	69.6 KB	Precompute summaries used by the UI instead of shipping full raw records
packages/game-core/src/data/skills/universal.json	51.1 KB	Consider generated indexes or server-side packaging by feature set.
packages/game-core/src/data/skills/class_abilities.json	37.4 KB	Consider generated indexes or server-side packaging by feature set.
packages/game-core/src/data/skills/race_abilities.json	32.8 KB	Consider generated indexes or server-side packaging by feature set.

Closing Notes

What to do next

The codebase does not read like a project in crisis. It reads like a project that has accumulated enough successful systems that the first generation of architecture is now carrying too much at once. That is a good problem to have, because the next step is not reinvention. It is refinement.

If the team starts with a narrow, measured wave focused on GameUI, useNarrative, lint scope, and chunk telemetry, it can create immediate momentum. From there, combat decomposition and content compilation become much easier to justify and much safer to ship.

- Start with visibility, not surgery.
- Reduce orchestration surfaces before chasing micro-optimizations.
- Compile or lazily load static content wherever feature boundaries allow.
- Use deterministic replay and bundle budgets as proof that the work is helping.

Report generated from the repository state observed on March 10, 2026. All recommendations are intended as implementation-ready guidance rather than abstract best practices.